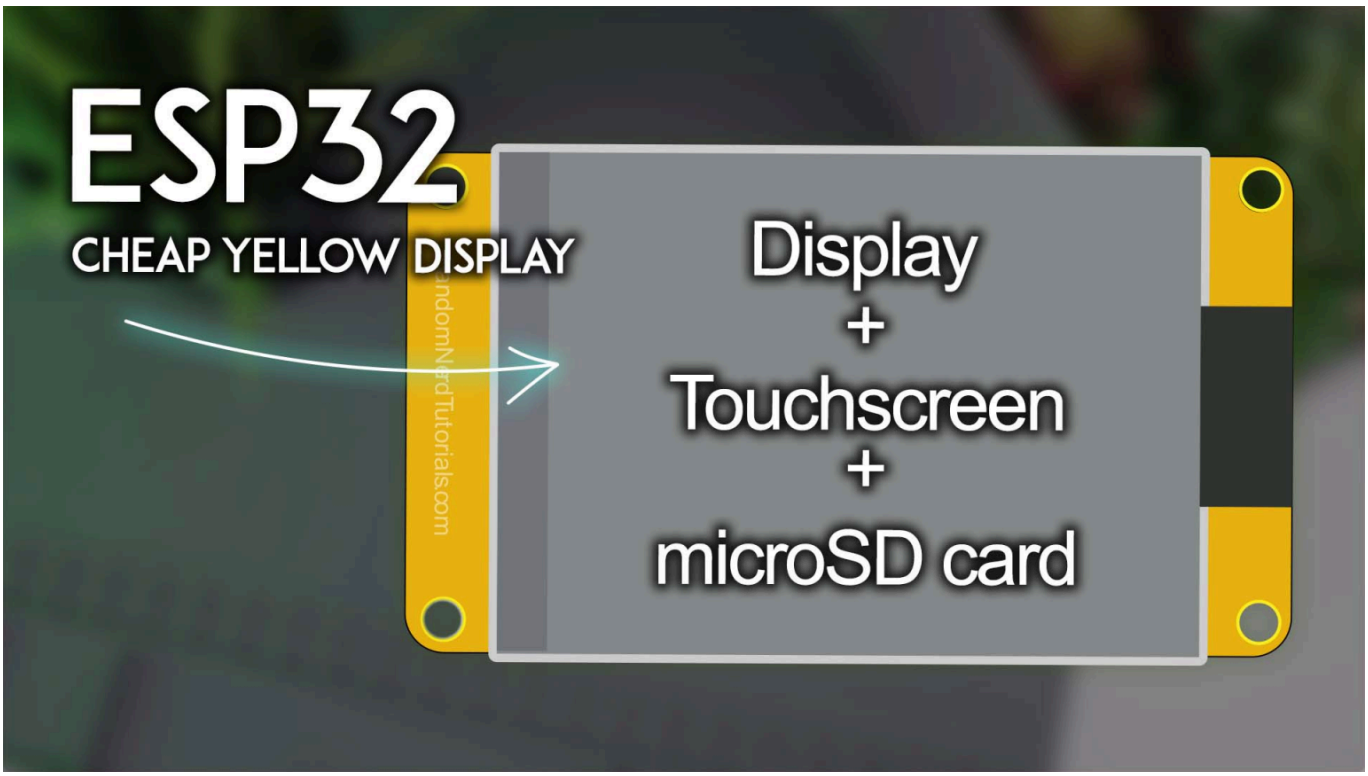
ESP32 CYD: Use Display, Touchscreen and MicroSD Card Simultaneously

In this guide, we'll show you how to use the display, touchscreen, and microSD card all at once with the ESP32 CYD (Cheap Yellow Display) board. We'll build a simple example that shows the current touch points on the screen while simultaneously saving them to the microSD card.



Although this is a basic example, it provides a solid foundation for understanding how to combine these features. Using all three functions simultaneously on the CYD board is not as straightforward as it seems due to hardware/library limitations related to the SPI communication protocol. This approach was tested on the ESP32-2432S028R – 2.8 inch 240×320 with a resistive touchscreen.

New to the ESP32 CYD board? Get Started with this guide: Getting Started with

Prerequisites

Before proceeding, make sure you follow the next prerequisites. **You must follow all steps, otherwise, your project will not work.**

1) Install ESP32 Boards in Arduino IDE



We'll program the ESP32 using Arduino IDE. Make sure you have the ESP32 boards installed. Follow the next tutorial:

- Installing ESP32 Board in Arduino IDE 2 (Windows, Mac OS X, Linux)

2) Install TFT and LVGL Libraries

LVGL (Light and Versatile Graphics Library) is a free and open-source graphics library that provides a wide range of easy-to-use graphical elements for your microcontroller projects that require a graphical user interface (GUI).

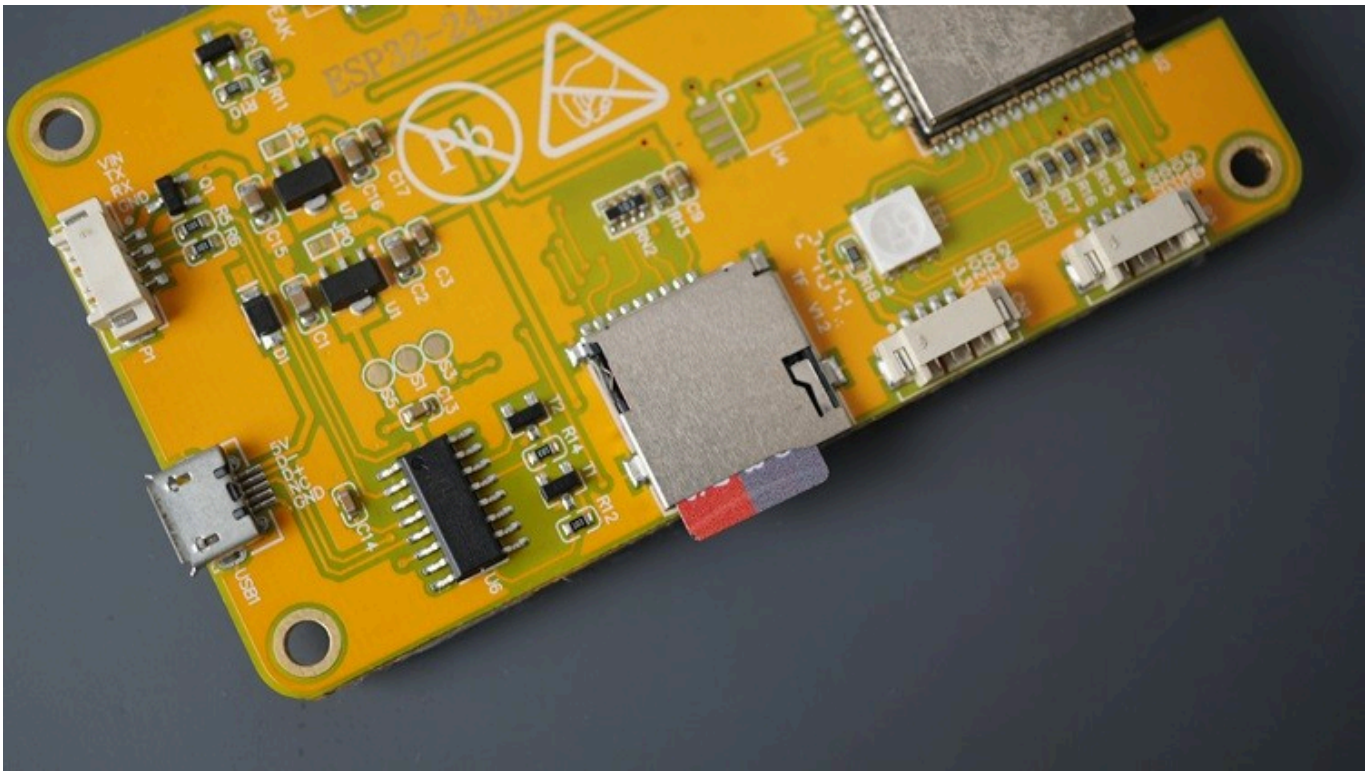


Follow the next tutorial to install and configure the required libraries to use LVGL for the ESP32 Cheap Yellow Display using Arduino IDE.

- **Get started with LVGL using the ESP32 Cheap Yellow Display Board**

The MicroSD Card and SPI Communication

Before diving into the example, it's important to review some key points about the SPI communication protocol on ESP32 boards, particularly the CYD.



The ESP32 supports two separate SPI buses: VSPI and HSPI. These are the default pins for VSPI and HSPI.

SPI	MOSI	MISO	SCLK	CS
VSPI	GPIO 23	GPIO 19	GPIO 18	GPIO 5
HSPI	GPIO 13	GPIO 12	GPIO 14	GPIO 15

> Recommended reading: ESP32 SPI Communication: Set Pins, Multiple SPI Bus Interfaces, and Peripherals (Arduino IDE).

The microSD card on the CYD uses the default VSPI pins:

MicroSD card	GPIO
MOSI	GPIO 23
MISO	GPIO 19
SCLK	GPIO 18
CS	GPIO 5

Recommended reading: ESP32 Guide for MicroSD Card Module (Arduino IDE).

Shared SPI Bus

Both the display and touchscreen use the SPI communication protocol. In our examples, the display library is configured to use the HSPI bus, while the touchscreen library uses the VSPI bus. This becomes problematic because the **microSD card also uses the VSPI bus (on the default hardware pins), creating a conflict**.

At first glance, this may not seem like a problem. After all, multiple devices can share the same SPI bus if each one has its own CS (Chip Select) pin.

HOWEVER, the issue arises with the touchscreen, which continuously listens for touch events. Temporarily disabling it while accessing the microSD card isn't ideal. Additionally, many users report this approach doesn't work reliably.

Software SPI for the touchscreen

So, what can we do about this?

We know that the ESP32 allows the use of Software SPI (SoftSPI) on any available pins. To solve the conflict, we can use a modified version of the `XPT2046_Touchscreen` library that communicates over SoftSPI instead of hardware

Download the XPT2046_TouchscreenSOFTSPI library

We'll call this modified version of the library `XPT2046_TouchscreenSOFTSPI`. This library also needs the `SoftSPI` library, which we'll also install locally.

- **Click here to download all the project files** (including the main sketch)

This link should download a *.zip* folder with the following files:

- XPT2046_TouchscreenSOFTSPI.cpp
- XPT2046_TouchscreenSOFTSPI.h
- SoftSPI.cpp
- SoftSPI.h
- The Arduino sketch

Unzip the folder.

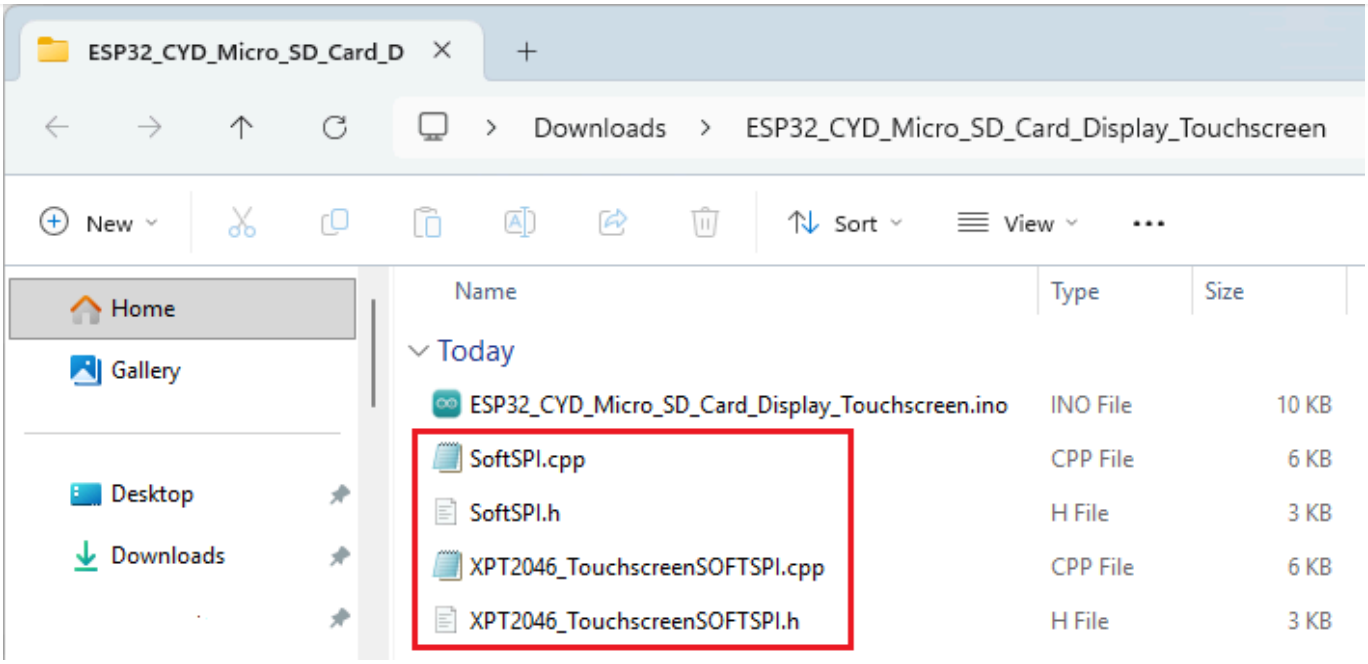
Moving the Library Files to the Sketch Folder

The previous folder already contains all the files to run the example. However, if you're starting a project from scratch, you'll need to move the library files to the sketch folder, as we'll explain next.

1) Open Arduino IDE. Save the current sketch folder (even if empty). File > Save as… to save it to a desired location.

2) Go to **Sketch** > **Show Sketch Folder**. This will open your current sketch folder. You should move those four library files into this folder.

3) Your Arduino sketch folder should look like this.



4) After moving the files, restart Arduino IDE.

Example Sketch (Display + Touchscreen + MicroSD Card)

After having all the required library files in the sketch folder, you can write your Arduino sketch. This is the example that we'll be running.

```
  // Set the callback function to read Touchscreen input
  lv_indev_set_read_cb(indev, touchscreen_read);

  lv_create_main_gui();
}

void loop() {
  lv_task_handler();  // let the GUI do its work
  lv_tick_inc(5);     // tell LVGL how much time has passed
  delay(5);           // let this time pass
 }
```

View raw code

How Does the Code Work?

This code is not much different from the example in this previous tutorial.

We've just added the lines of code to interface with the microSD card, save the touchpoints, and initialize the touchscreen using the modified library.

We'll take a look at the relevant parts of code to show how to use the display, touschscreen and microSD card simultaneously,

Import the XPT2046_TouchscreenSOFTSPI library

We import the modified version of the touchscreen library:

```
#include <XPT2046_TouchscreenSOFTSPI.h>
```

Create a SoftSPI Instance

```
SoftSPI *tsSoftSpi;
```

Create a Touchscreen Object

We create a new touchscreen object using the modified version of the library.

```
XPT2046_TouchscreenSOFTSPI touchscreen(XPT2046_CS, XPT2046_IRQ);
```

Import Libraries for the MicroSD Card

We import the `FS` and `SD` libraries, needed to handle files on the microSD card.

```
#include "FS.h"
#include "SD.h"
```

MicroSD Card SPI Pins and Bus

We define the pins for the microSD card (default VSPI pins), and create a new SPI instance on VSPI with those pins.

```
#define SD_MISO  19
#define SD_MOSI  23
#define SD_SCK  18
#define SD_CS  5
SPIClass Sd_spi = SPIClass(VSPI);
```

MicroSD Card Functions

We then have several defined functions to interact with the microSD card. We won't use them all, but we're leaving them in this example so that you know which functions

```
void listDir(fs::FS &fs, const char * dirname, uint8_t levels){
  Serial.printf("Listing directory: %s\n", dirname);

  File root = fs.open(dirname);
  if(!root){
    Serial.println("Failed to open directory");
    return;
  }
  if(!root.isDirectory()){
    Serial.println("Not a directory");
    return;
  }

  File file = root.openNextFile();
  while(file){
    if(file.isDirectory()){
      Serial.print("  DIR : ");
      Serial.println(file.name());
      if(levels){
        listDir(fs, file.name(), levels -1);
      }
    } else {
      Serial.print("  FILE: ");
      Serial.print(file.name());
      Serial.print("  SIZE: ");
      Serial.println(file.size());
    }
    file = root.openNextFile();
  }
}

void createDir(fs::FS &fs, const char * path){
  Serial.printf("Creating Dir: %s\n", path);
  if(fs.mkdir(path)){
```

```
  } else {
    Serial.println("mkdir failed");
  }
}

void removeDir(fs::FS &fs, const char * path){
  Serial.printf("Removing Dir: %s\n", path);
  if(fs.rmdir(path)){
    Serial.println("Dir removed");
  } else {
    Serial.println("rmdir failed");
  }
}

void readFile(fs::FS &fs, const char * path){
  Serial.printf("Reading file: %s\n", path);

  File file = fs.open(path);
  if(!file){
    Serial.println("Failed to open file for reading");
    return;
  }

  Serial.print("Read from file: ");
  while(file.available()){
    Serial.write(file.read());
  }
  file.close();
}

void writeFile(fs::FS &fs, const char * path, const char * message)
  Serial.printf("Writing file: %s\n", path);

  File file = fs.open(path, FILE_WRITE);
```

```
    return;
  }
  if(file.print(message)){
    Serial.println("File written");
  } else {
    Serial.println("Write failed");
  }
  file.close();
}

void appendFile(fs::FS &fs, const char * path, const char * message
  Serial.printf("Appending to file: %s\n", path);

  File file = fs.open(path, FILE_APPEND);
  if(!file){
    Serial.println("Failed to open file for appending");
    return;
  }
  if(file.print(message)){
      Serial.println("Message appended");
  } else {
    Serial.println("Append failed");
  }
  file.close();
}

void deleteFile(fs::FS &fs, const char * path){
  Serial.printf("Deleting file: %s\n", path);
  if(fs.remove(path)){
    Serial.println("File deleted");
  } else {
    Serial.println("Delete failed");
  }
}
```

Additionally, we recommend taking a look at our microSD card guide for more details about interfacing a microSD card with the ESP32.

- ESP32: Guide for MicroSD Card Module (Arduino IDE)

setup()

In the `setup()`, we need to add the following lines to initialize the microSD card:

```
Serial.print("\nInitializing SD card...\n");

Sd_spi.begin(SD_SCK, SD_MISO, SD_MOSI, SD_CS);

if (!SD.begin(SD_CS, Sd_spi, 55000000)) {
  Serial.println("Card Mount Failed");
}
```

We also print information about the microSD card type, size, and currently saved files.

```
uint8_t cardType = SD.cardType();

if(cardType == CARD_NONE){
  Serial.println("No SD card attached");
  return;
}

Serial.print("SD Card Type: ");
if(cardType == CARD_MMC){
  Serial.println("MMC");
} else if(cardType == CARD_SD){
  Serial.println("SDSC");
} else if(cardType == CARD_SDHC){
```

```
  } else {
    Serial.println("UNKNOWN");
  }

    uint64_t cardSize = SD.cardSize() / (1024 * 1024);
    Serial.printf("SD Card Size: %lluMB\n", cardSize);

    listDir(SD, "/", 0);
```

Then, we create a new file on the microSD card called `touch_point.txt` to save the touch points, if it doesn't already exist. We also create the headers for the file.

```
  // If the touch_points.txt file doesn't exist
  //create a file on the SD card and write the header
  File file = SD.open("/touch_points.txt");
  if(!file) {
    Serial.println("touch_points.txt file doesn't exist");
    Serial.println("Creating file...");
    writeFile(SD, "/touch_points.txt", "x,y,z\r\n");
  }
  else {
    Serial.println("touch_points.txt file already exists");
  }
  file.close();
```

Still in the `setup()`, we need to initialize the touchscreen using Software SPI.

```
  // Create a Softspi Object and set its SPI pins
  tsSoftSpi = new SoftSPI (XPT2046_MOSI, XPT2046_MISO, XPT2046_CLK);

  // Start the SPI for the touchscreen and init the touchscreen
  touchscreen.begin(tsSoftSpi);
```

Saving the Touch Points to the MicroSD Card

Another change we need to make is to add the lines of code to save the touchpoints to the microSD card. We can do that on the `touchscreen_read()` function.

```
// Get the Touchscreen data
void touchscreen_read(lv_indev_t * indev, lv_indev_data_t * data) {
  // Checks if Touchscreen was touched, and prints X, Y and Pressur
  if(touchscreen.tirqTouched() && touchscreen.touched(tsSoftSpi)) {
    // Get Touchscreen points
    TS_Point p = touchscreen.getPoint(tsSoftSpi);
    // Calibrate Touchscreen points with map function to the correc
    x = map(p.x, 200, 3700, 1, SCREEN_WIDTH);
    y = map(p.y, 240, 3800, 1, SCREEN_HEIGHT);
    z = p.z;

    data->state = LV_INDEV_STATE_PRESSED;

    // Set the coordinates
    data->point.x = x;
    data->point.y = y;

    // Print Touchscreen info about X, Y and Pressure (Z) on the Se
    Serial.print("X = ");
    Serial.print(x);
    Serial.print(" | Y = ");
    Serial.print(y);
    Serial.print(" | Pressure = ");
    Serial.print(z);
    Serial.println();
    String touch_data = "X = " + String(x) + "\nY = " + String(y)
                      + "\nZ = " + String(z);
    lv_label_set_text(text_label_touch, touch_data.c_str());
```

```
      appendFile(SD, "/touch_points.txt", sdMessage.c_str());
    }
    else {
      data->state = LV_INDEV_STATE_RELEASED;
    }
  }
```

After detecting touch and saving the data points and pressure on the `x`, `y`, and `z` variables, we can save them to the file on the microSD card as follows.

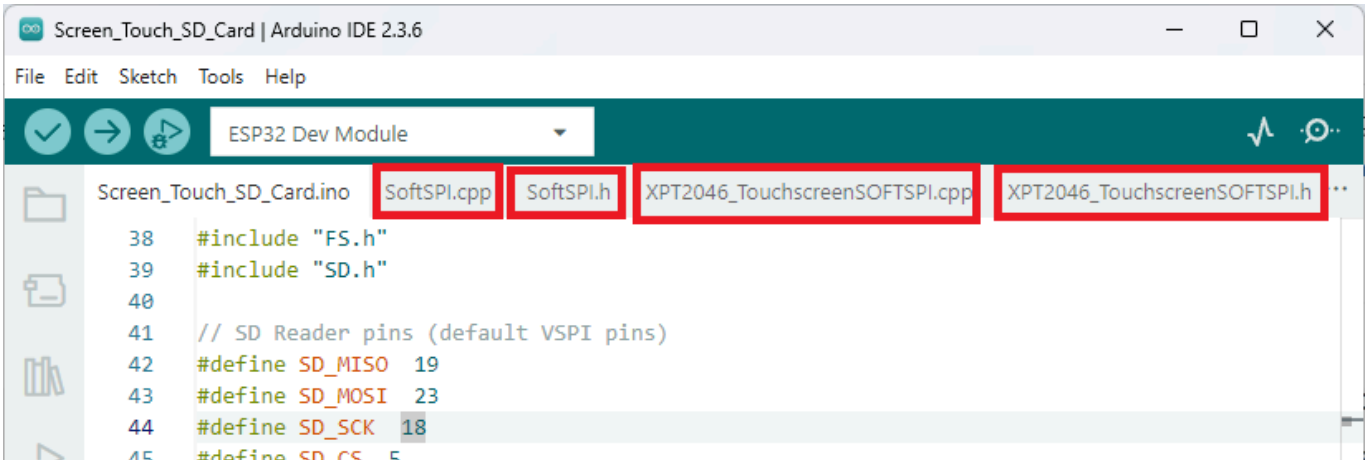
```
String sdMessage = String(x) + "," + String(y) + "," + String(z) +
appendFile(SD, "/touch_points.txt", sdMessage.c_str());
```

Demonstration

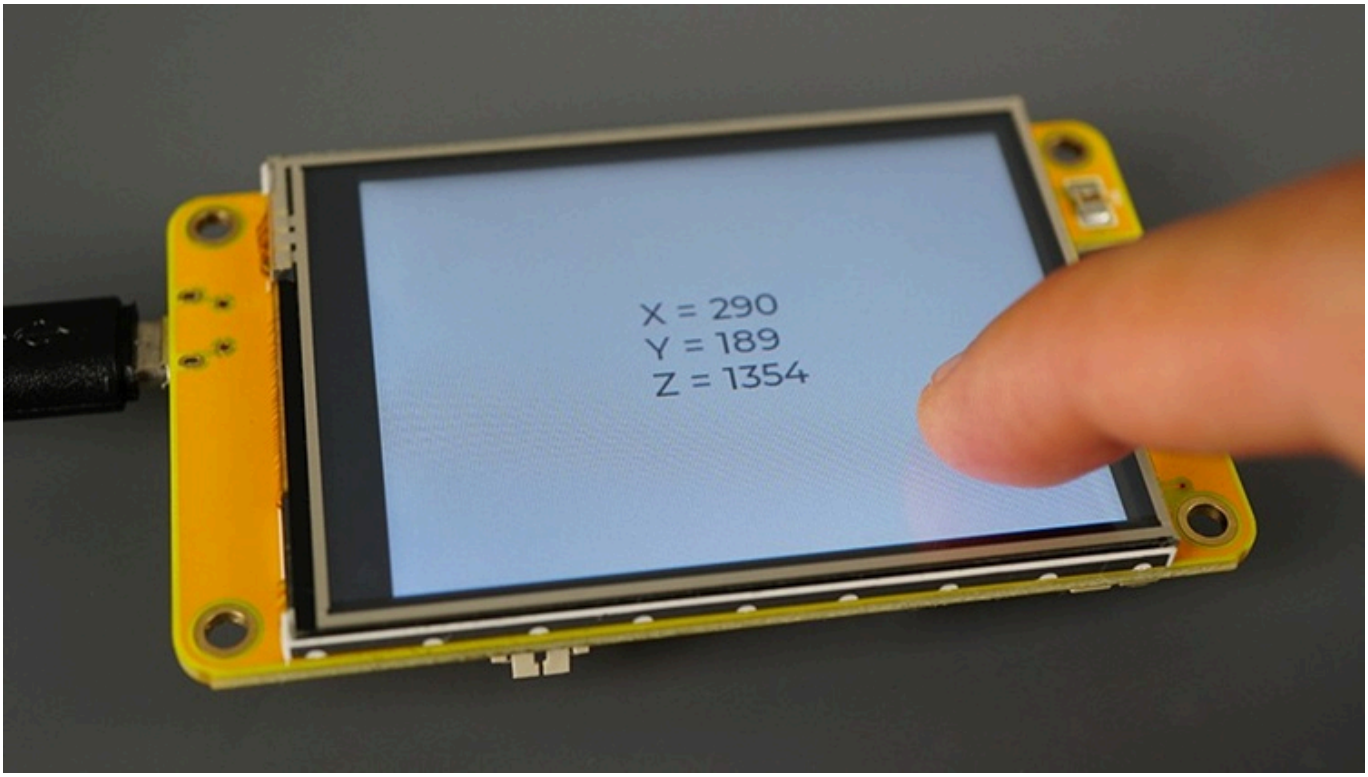
Now, you can upload the code to your CYD board.

Make sure you have the required files to use `SoftSPI` for the touchscreen in your sketch folder.

Those files should be available as tabs in your Arduino IDE project (if not, it means they are not in the correct location).

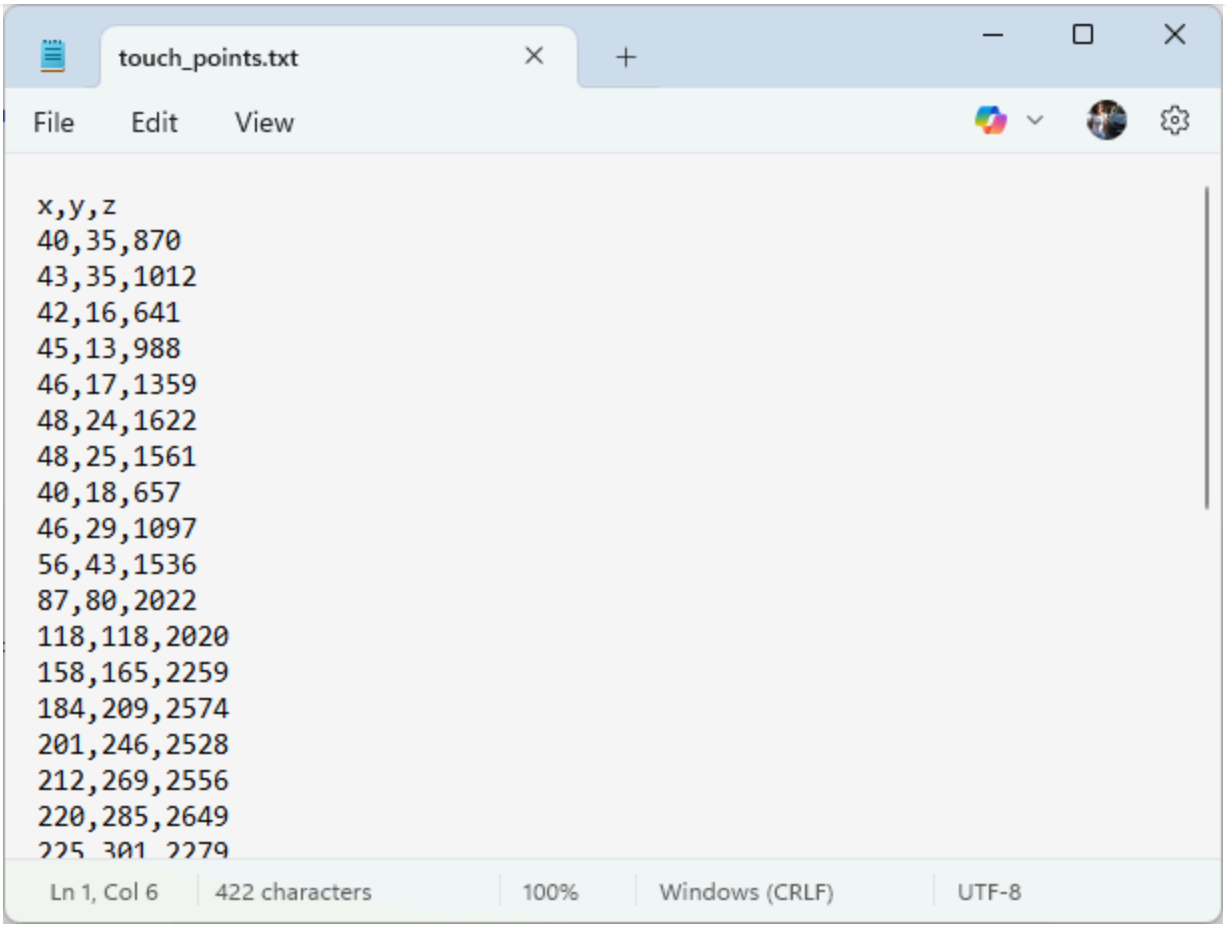


After uploading the code, touch on a few random points on the screen. The screen will show the touched points. At the same time, those points will be saved on the microSD card.



After touching a few number of points, you can remove the microSD card from the CYD and insert it into your computer.

It should have a file called `touch_points.txt` . You can open that file to check which points were saved to the microSD card.



Now, you know which tricks you need to do to be able to use the microSD card, touchscreen, and display simultaneously.

When You Don't Need to Use the Modified Touchscreen Library?

Finally, I would like to clarify that you only need to use the modified touchscreen library when using these three things simultaneously: screen, touchscreen, and microSD card.

If the microSD card and the touchscreen are not used on the same project, there's no need to use the modified version of the touchscreen library.

Wrapping Up

If you're using the ESP32-2432S028R – 2.8 inch 240×320 with a resistive

card, touchscreen, and the display simultaneously because all three components use the SPI communication protocol.

In this guide, we've shown you a simple trick you can use if you want to use the three features together in your project.

We hope this quick guide was helpful.

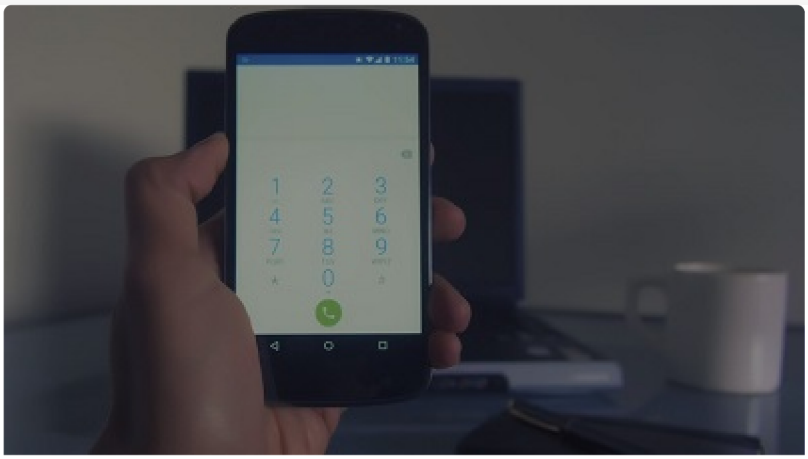
If you want to learn more about controlling the CYD display and creating awesome ESP32 projects with a nice graphical user interface, check out our eBook:

- **Learn LVGL: Build GUIs for ESP32 Projects**

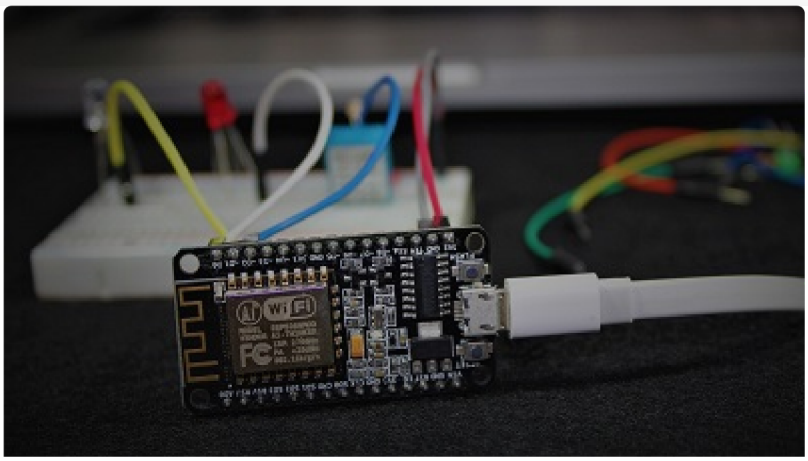
SMART HOME with Raspberry Pi, ESP32, ESP8266 [eBook]

Learn how to build a home automation system and we'll cover the following main subjects: Node-RED, Node-RED Dashboard, Raspberry Pi, ESP32, ESP8266, MQTT, and InfluxDB

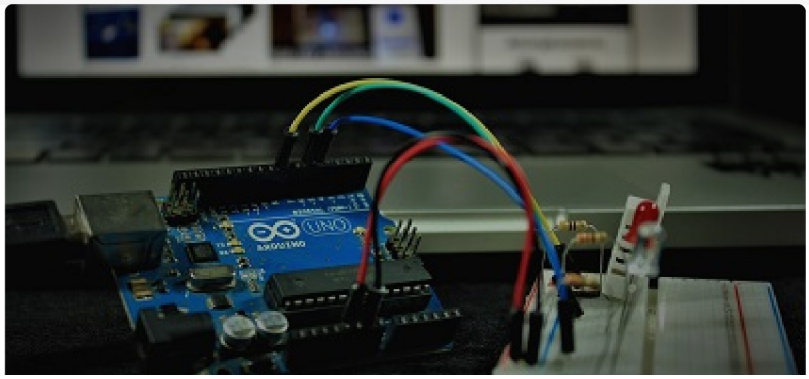
Recommended Resources



Build a Home Automation System from Scratch » With Raspberry Pi, ESP8266, Arduino, and Node-RED.



Home Automation using ESP8266 eBook and video course » Build IoT and home automation projects.



Arduino Step-by-Step Projects » Build 25 Arduino projects with our course, even with no prior experience!

What to Read Next…

ESP32 Plot Sensor Readings in Charts (Multiple Series)

MicroPython: Send Emails with the ESP32/ESP826

Enjoyed this project? Stay updated by subscribing our newsletter!

Your Email Address

SUBSCRIBE

3 thoughts on “ESP32 CYD: Use Display, Touchscreen and MicroSD Card Simultaneously”



Paulo Robalo

March 26, 2026 at 5:35 pm

Hello,

this is all very good and educational stuff, but the really fun part of these

INPUTS and OUTPUTS. And in this department, the ESP32-CYD doesn't shine. The number of I/Os is very limited, perhaps insufficient for any practical project needing to control relays, sensors, etc. For simple GUI and WiFi/Bluetooth communications, this is a pretty good module.
Do you know a practical way of adding more I/Os, perhaps using an I/O expander (I've used three MCP23008 in a recent project involving an ESP32-S3) ? or do you know a better way?
It would be very interesting to deepen this subject and create some projects using more I/Os – if possible or feasible, we always have ESP32 modules with more I/Os, but no included LCD. Just a suggestion.

Thanks and keep up the good work!

Reply



Jay
March 30, 2026 at 10:50 am

Share spi screen and SD card and put touch on its own spi. That's what I did and everything works fine with standard libraries

Reply

fxarax
May 11, 2026 at 2:28 am

I can suggest this different solution with standard libraries:



Reply

Leave a Comment

Name *

Email *

Website

Notify me of follow-up comments by email.

Notify me of new posts by email.

Post Comment

Affiliate Disclosure: *Random Nerd Tutorials is a participant in affiliate advertising programs designed to provide a means for us to earn fees by linking to Amazon, eBay, AliExpress, and other sites.*



Learn ESP32 with Arduino IDE eBook »
Complete guide to program the ESP32 with Arduino IDE!



SMART HOME with Raspberry Pi, ESP32, and ESP8266 » learn how to build a complete home automation system.



Learn Raspberry Pi Pico/Pico W with MicroPython » The complete getting started guide to get the most out of the the Raspberry Pi Pico/Pico W (RP2040) microcontroller board using MicroPython programming language.



🔥 **Learn LVGL: Build GUIs for ESP32 Projects »** Learn how to build Graphical User Interfaces (GUIs) for ESP32 Projects using LVGL (Light Versatile Graphics Library) with the Arduino IDE.

Copyright © 2013-2026 · RandomNerdTutorials.com · All Rights Reserved